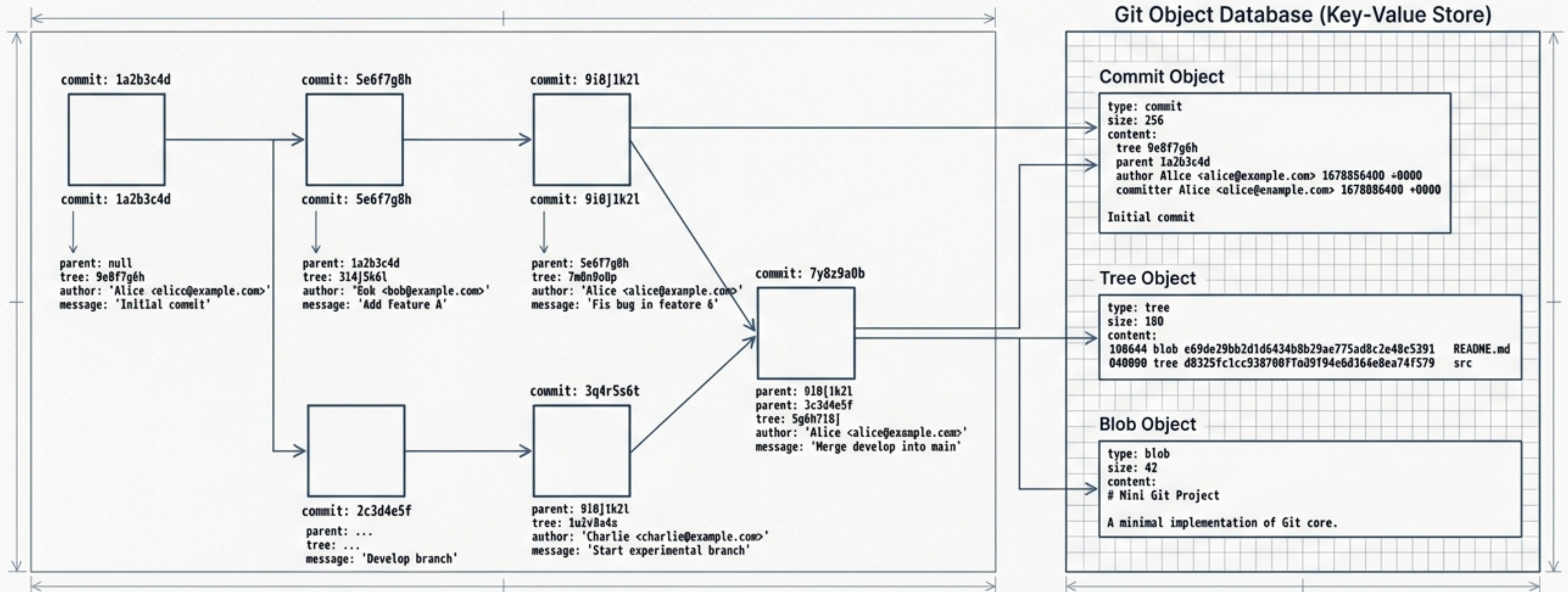


마법에서 기계로: Mini Git 코어 엔진 해체 분석

일상적인 git 명령어 뒤에 숨겨진 자료구조와 알고리즘의 실체
일상적인 git 명령어 뒤에 숨겨진 자료구조와 알고리즘의 실체



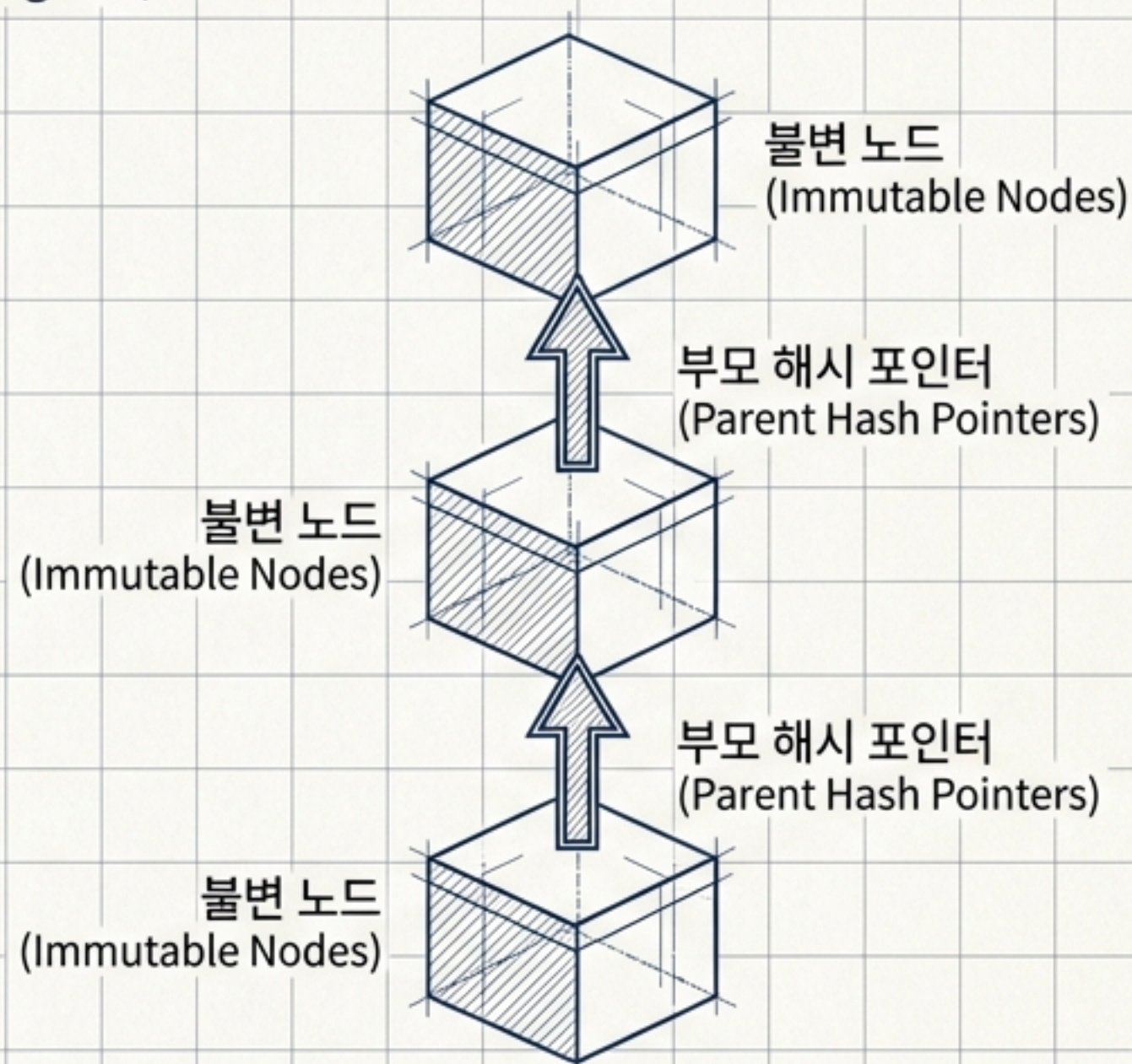
(Illustration)



우리의 일상 (블랙박스)

대부분의 개발자는 습관적으로 명령어를 입력하지만,
이면의 자료구조를 인지하지 못합니다.

(Diagram)



실체 대면하기

외부 라이브러리의 마법을 걷어내고,
불변 노드와 포인터로 구성된 실체를 직접 만져봅니다.

✗ `pip install networkx`

✗ `import pandas`

✗ `sorted()` / `list.sort()`

**Zero-
Dependency**



**Python 3.10+
Standard Library Only**

핵심 기술적 결정 1: Zero-Dependency 순수 구현

파이썬 내장 정렬 API와 외부 패키지를 전면 배제하고, 오직 순수 표준 라이브러리만으로 어떤 환경에서든 즉시 실행되는 엔진을 바닥부터 설계했습니다.

Engine Block

`models.py` : 도메인 엔티티

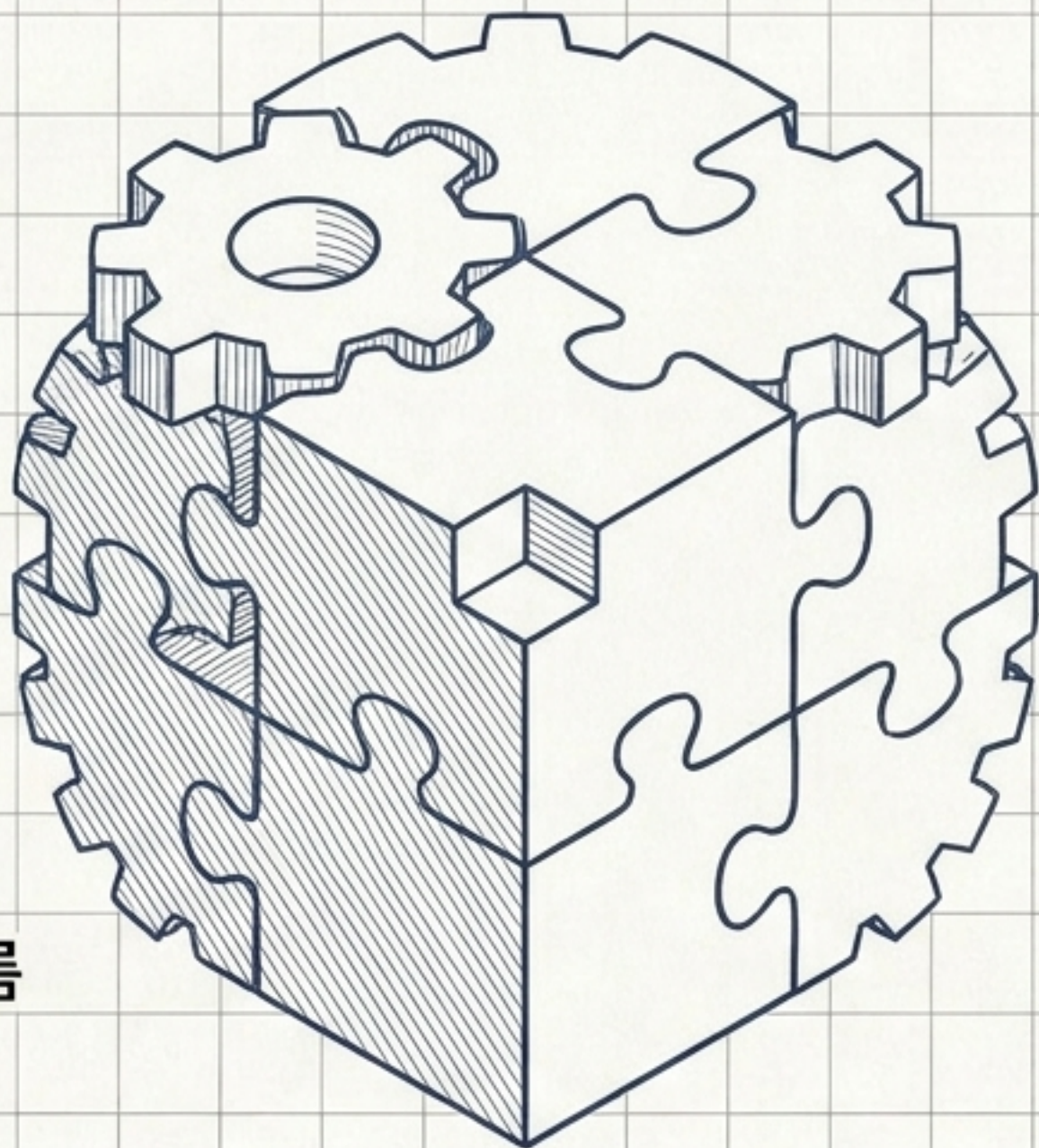
`sorting.py` : 정렬 엔진

`indexing.py` : 역색인 검색

`graph.py` : 그래프 탐색

`repository.py` : 비즈니스 흐름

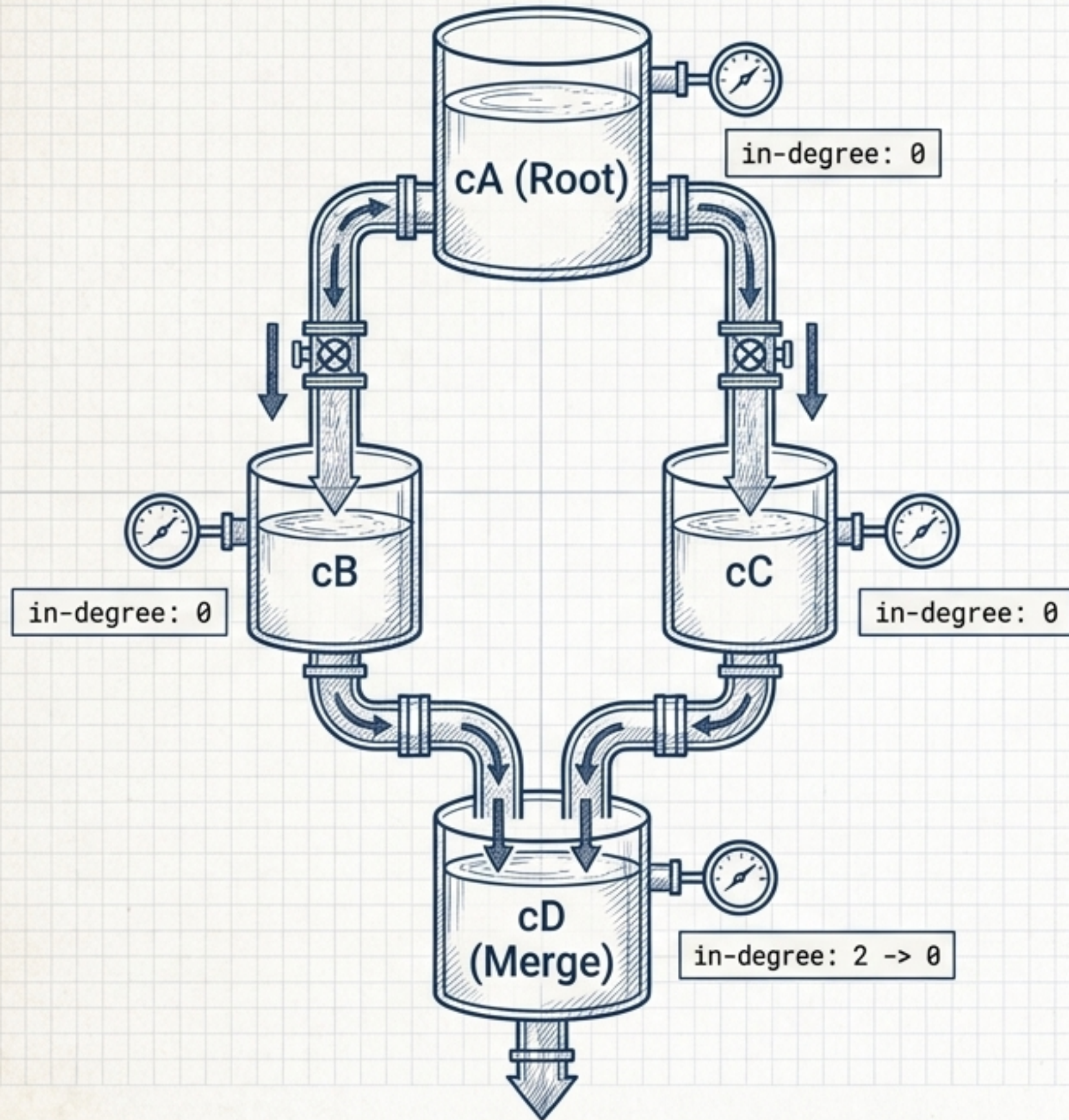
`cli.py` : REPL 인터페이스



단일 책임 원칙 (SRP) 기반 모듈 분리

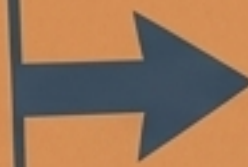
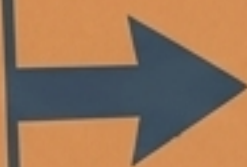
각 모듈은 톱니바퀴처럼 맞물려 돌아가되, 각자의 역할에만 완벽히 집중합니다.
데이터와 흐름 제어를 철저히 분리한 아키텍처입니다.

Fluid Dynamics / Valve System



심층 탐구 1: DAG 위상 정렬 (Topological Sort)

- 목표: 부모가 자식보다 무조건 먼저 출력되는 로그 보장
- Kahn's 알고리즘: 부모 커밋의 진입 차수(in-degree)가 0이 되어야만 실행 흐름이 다음 자식으로 흘러갑니다.

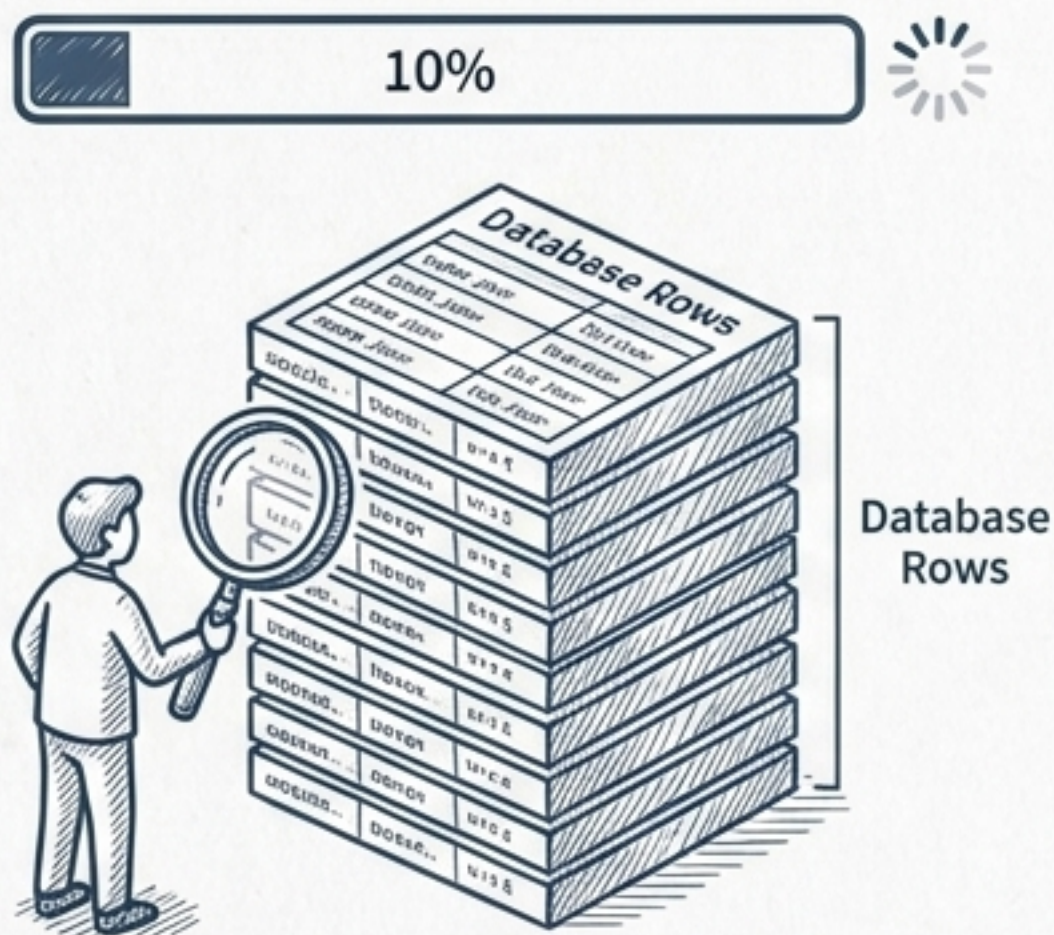


일상 비유: 라면 끓이기 레시피

계란을 먼저 풀고 나중에 물을 끓일 수는 없습니다. 부모 작업(선행 조건)이 완료되어야만 자식 작업이 시작될 수 있는 것. 이것이 커밋 그래프의 위상 정렬입니다.

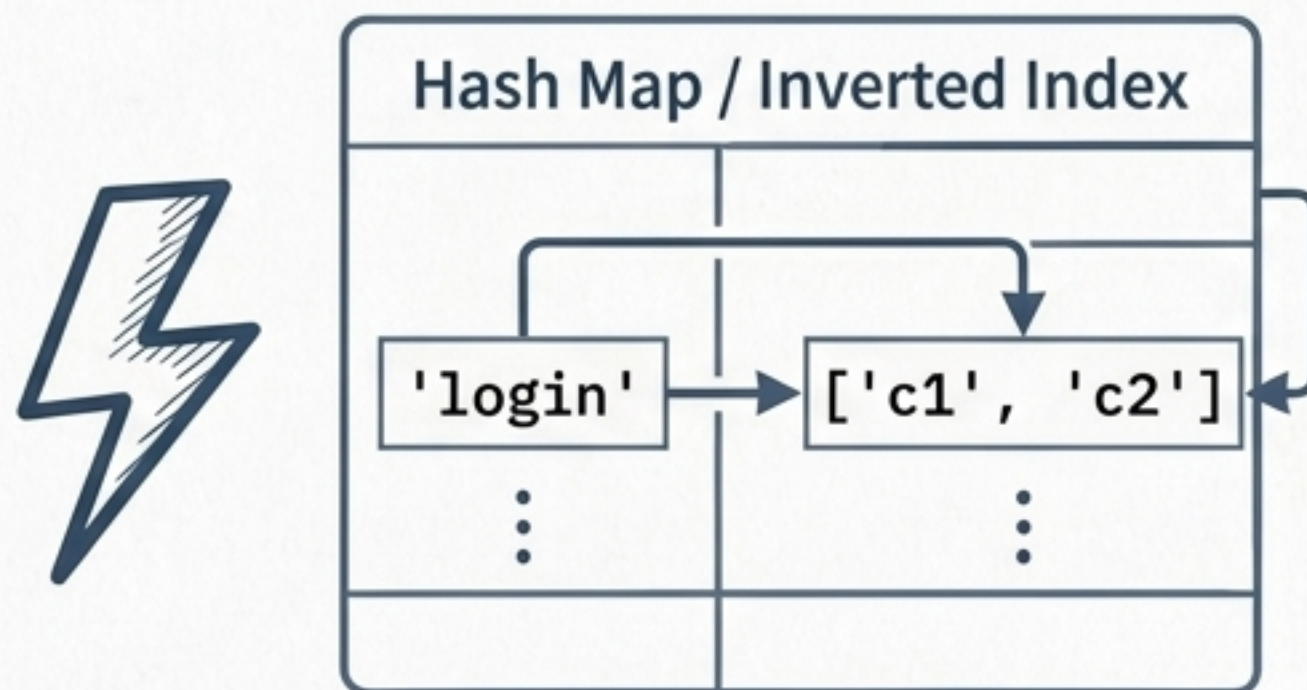
심층 탐구 2: 역색인(Inverted Index) 아키텍처

전체 순회 비효율 - $O(N)$

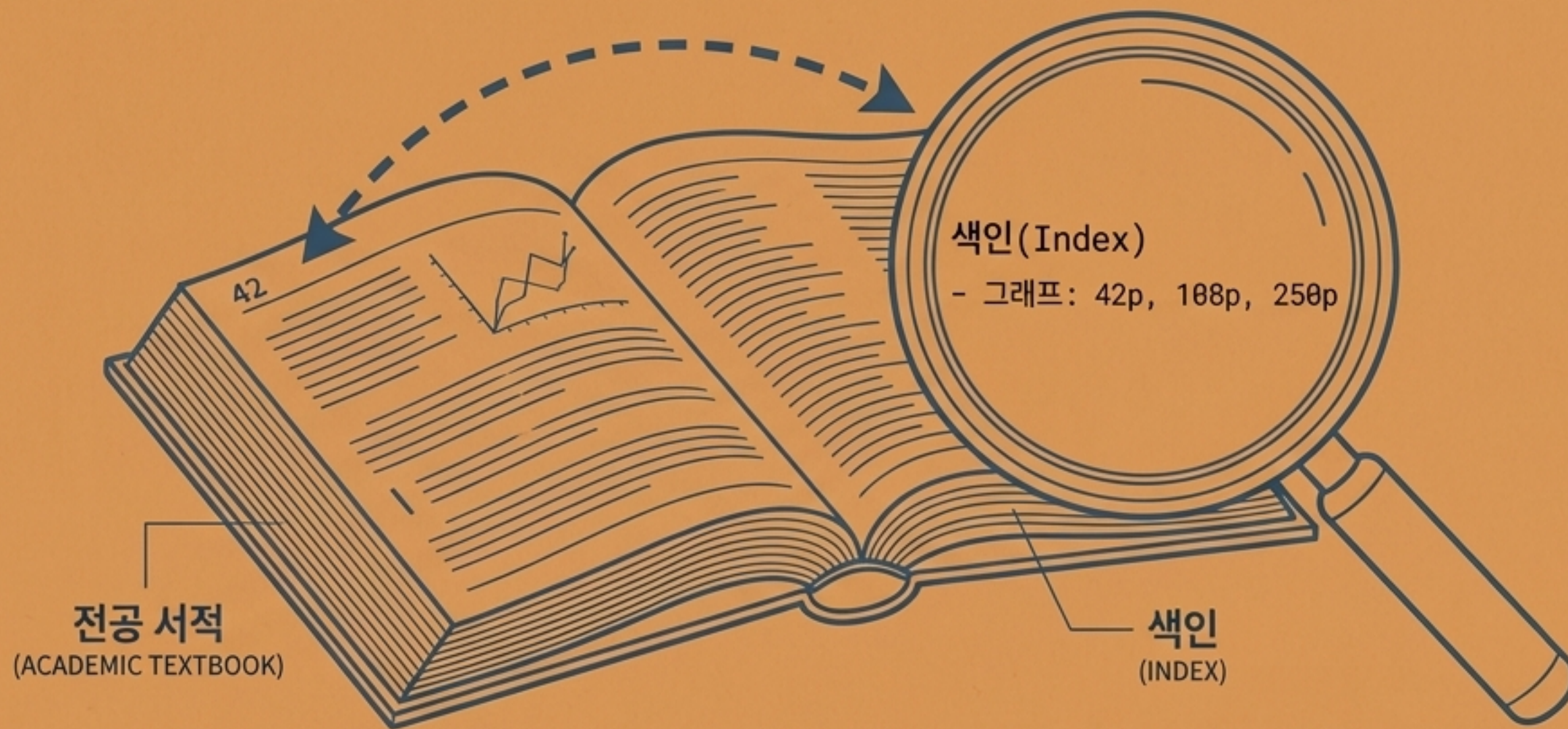


메시지 검색 시 모든 커밋을 순회하는 낭비 발생

초고속 역색인 검색 - $O(1)$

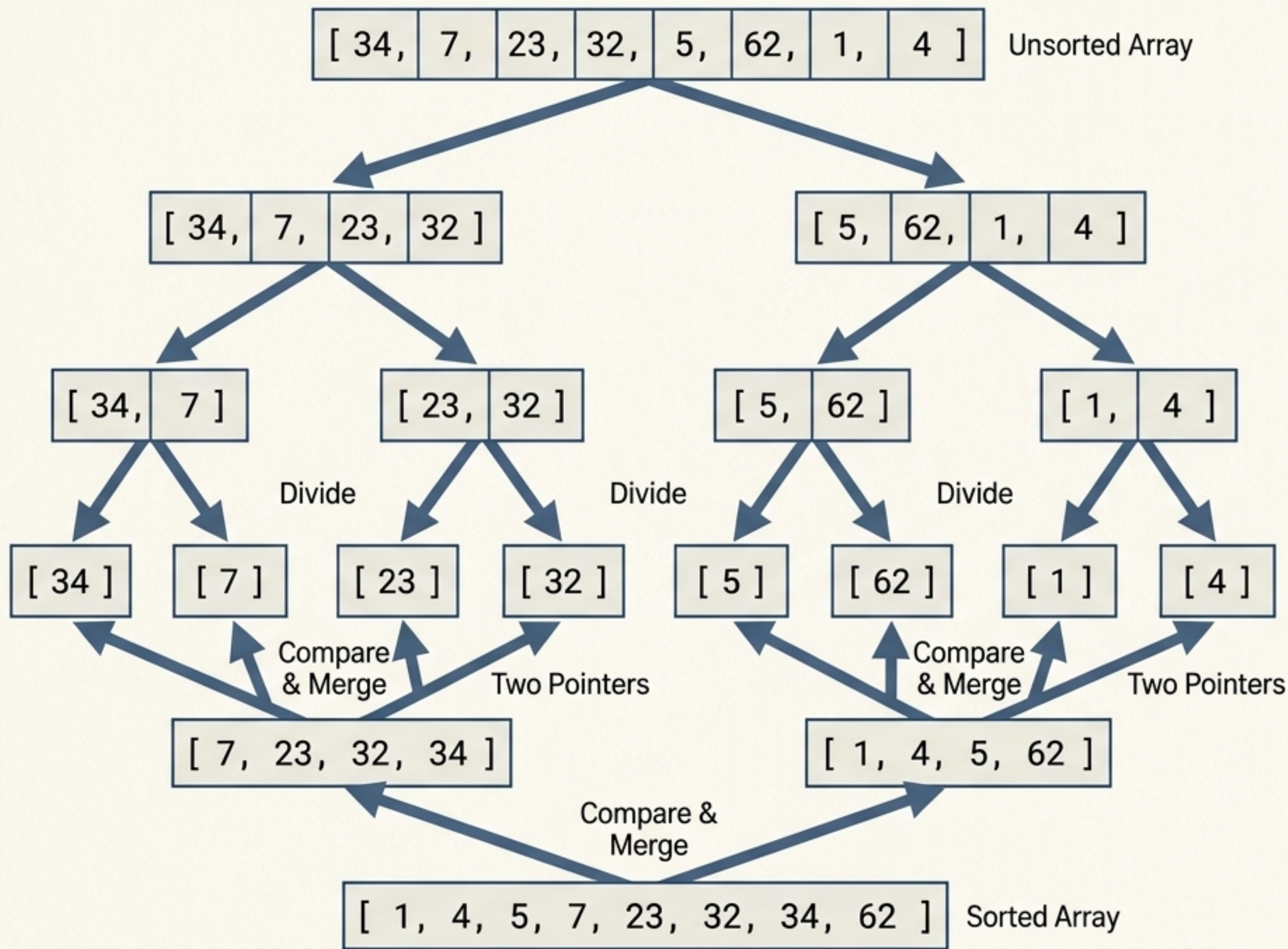


커밋 생성 시 토큰을 정규화하여 딕셔너리에
즉시 등록, 초고속 해시맵 룩업 구현



일상 비유: 전공 서적 맨 뒷장의 색인

‘그래프’라는 단어를 찾기 위해 1페이지부터 500페이지까지 전부 읽는 사람은 없습니다. 색인에서 단어를 찾고 해당 페이지로 즉시 점프하는 구조가 역색인의 본질입니다.



심층 탐구 3: 순수 수제 안정 병합 정렬 (Merge Sort)

파이썬 내장 API 금지 제약을 돌파하기 위해 바닥부터 직접 작성한 분할 정복 엔진입니다. 최악의 경우에도 $O(N \log N)$ 의 시간 복잡도를 엄격히 보장합니다.



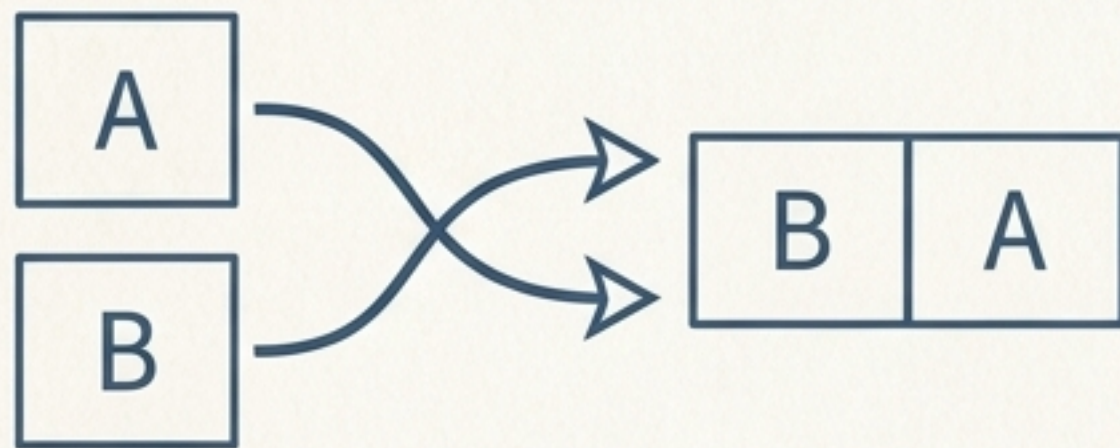
일상 비유: 시험지 두 묶음 합치기

미리 정렬된 두 묶음의 시험지가 있을 때, 전체를 다시 섞지 않고
맨 위 번호만 비교하며 차곡차곡 합치는 안정적인 분할 정복 작업입니다.

동률 발생 시 '안정성(Stability)'의 위력

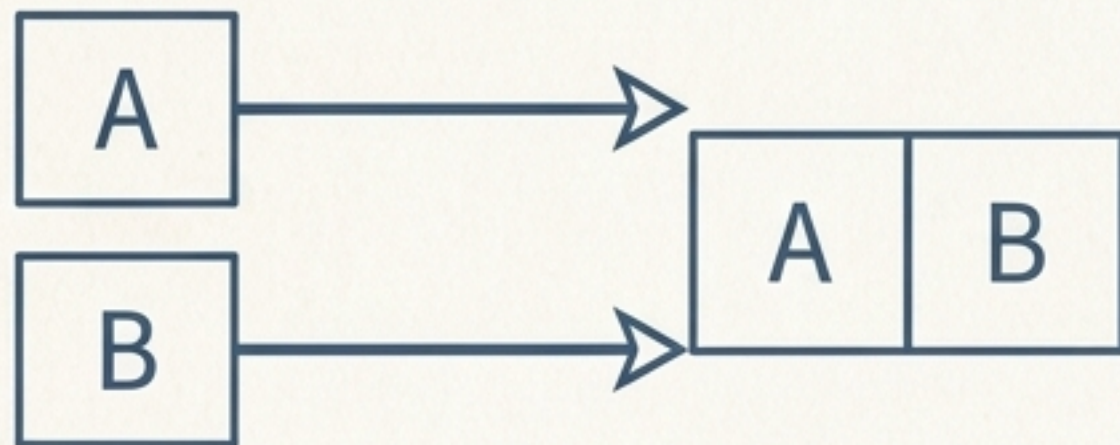
조건: 커밋 A와 B의 타임스탬프가 10.0s로 완전히 동일하게 충돌할 때

불안정 정렬
(Unstable Sort)

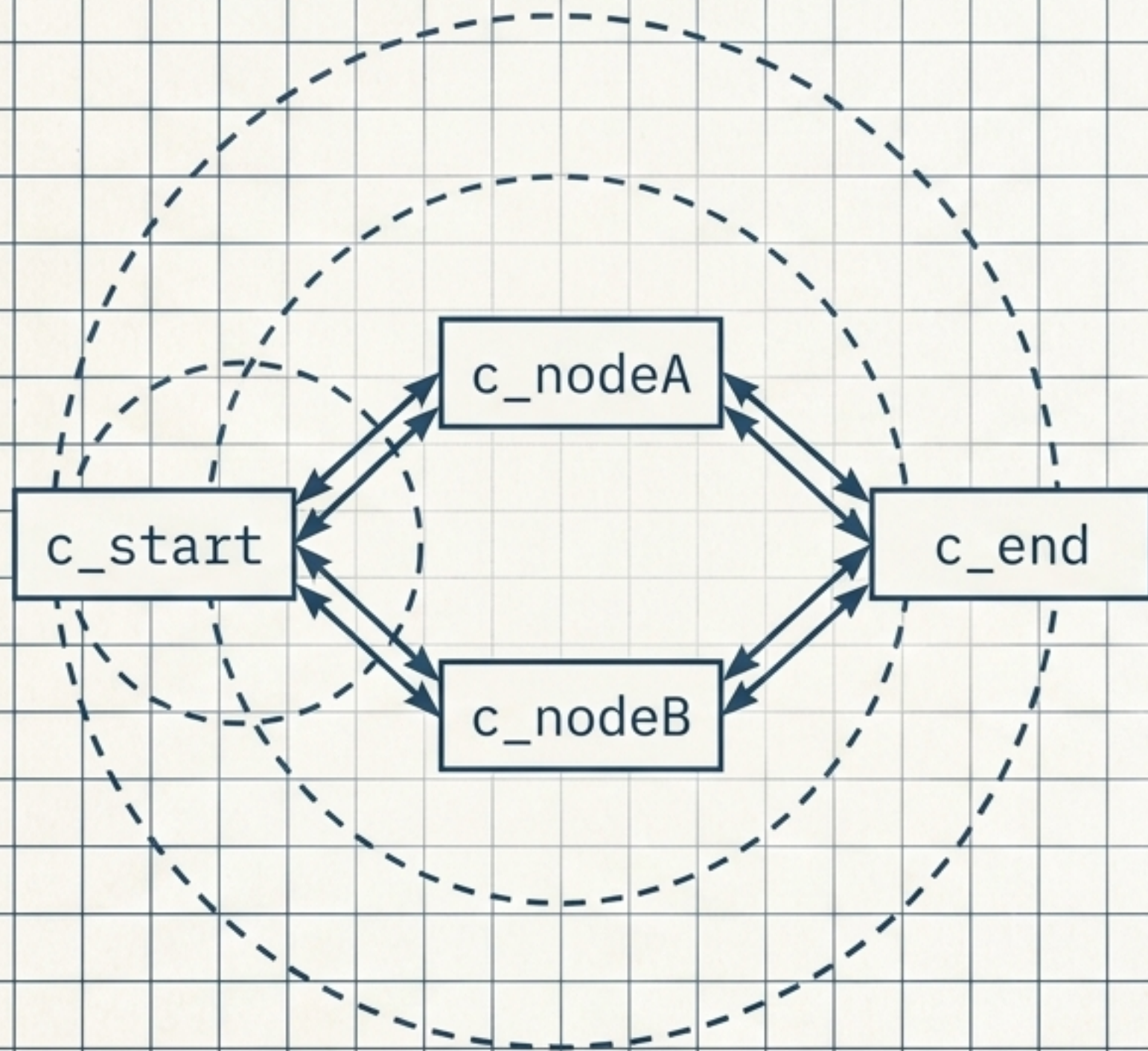


원본 역사 훼손:
순서가 뒤섞임

안정 병합 정렬
(Stable Merge Sort)

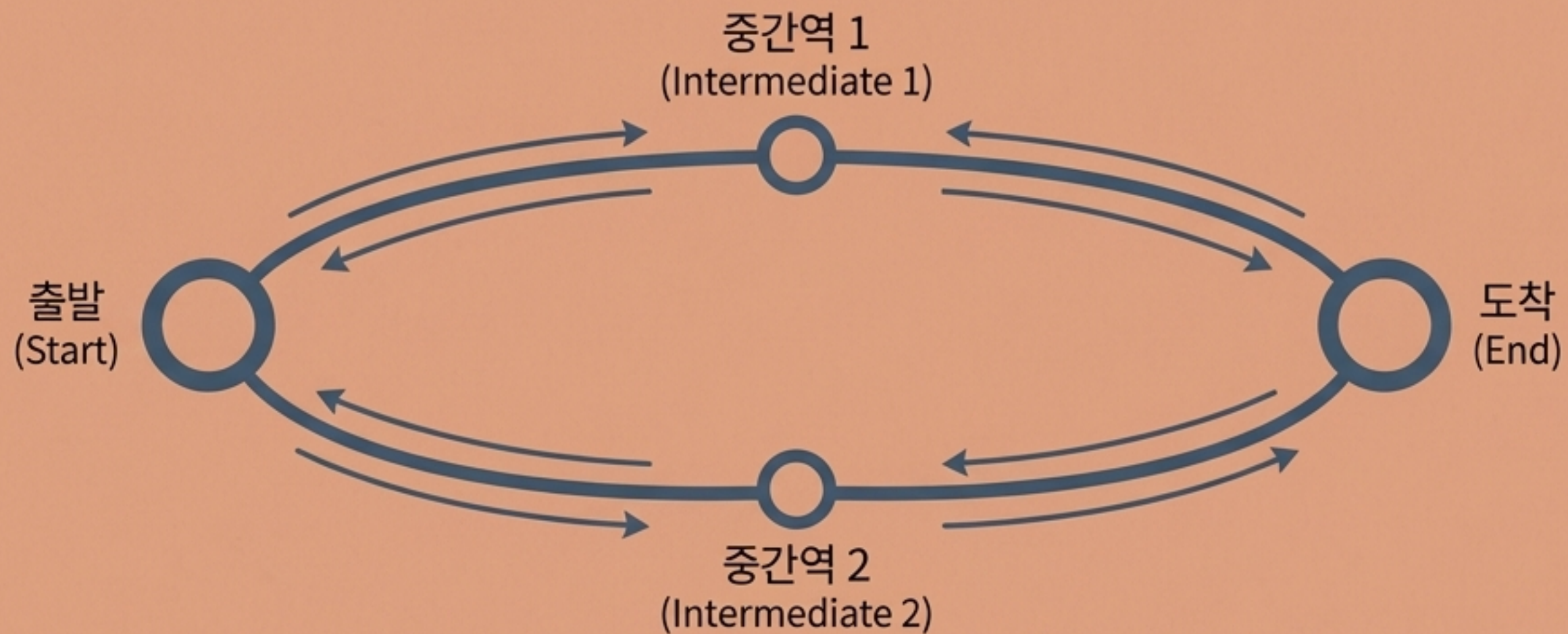


생성 순서 100% 보존:
원래 커밋 역사를
끝까지 지켜냄



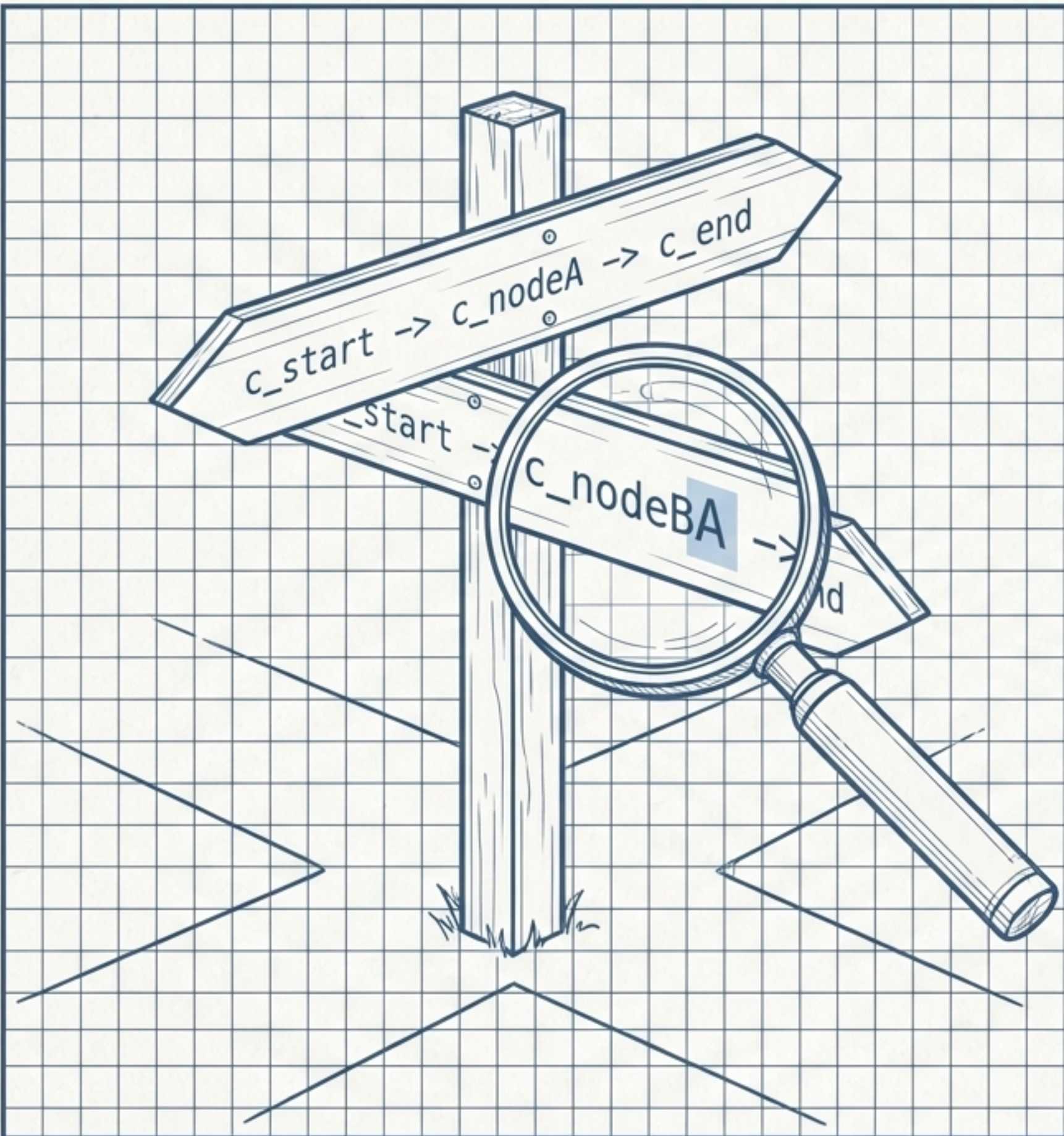
심층 탐구 4: 무방향 BFS 최단 경로 탐색

단방향 부모 연결을 양방향 통행 간선으로 간주합니다. 너비 우선 탐색(BFS)을 계층별로 수행하여 가장 적은 간선으로 도달하는 경로를 도출합니다.



일상 비유: 지하철 최소 환승 노선

역방향 탑승이 가능한 지하철에서, 가장 적은 역을 거쳐 목적지에 도착하는 노선을 찾는 것과 같습니다. 만약 거리가 똑같은 노선이 2개라면 어떻게 선택해야 할까요?



결정론적(Deterministic) 알고리즘의 엄밀함

- 동일한 최소 간선 경로가 여러 개일 때의 타이 브레이커 규칙.
- 실행 환경이나 우연에 휘둘리지 않고 해시 문자열의 사전순(Lexicographical)을 비교합니다.
- 항상 100% 동일한 결과를 보장하는 것이 진정한 소프트웨어 엔지니어링입니다.

결합: 0.004초의 코어 엔진 완성

외부 의존성 제약과 내장 API 금지라는 제약 속에서도 SRP 아키텍처가 결합하여, 28개의 테스트를 단 0.004초 만에 통과하는 견고한 시스템을 완성했습니다. 우리는 마법을 걷어내고 기계를 온전히 통제합니다.



$O(1)$
인덱스 검색



DAG 위상 정렬 로그



$O(N \log N)$
수제 정렬



결정론적 BFS 탐색